

Express.js: Day 2

Request and response objects, route matching, `res.send()`, and practical request flow.

Day 2 goal

Understand what `req` and `res` represent, how Express exposes request information, what `res.send()` does, and why a route executes only when the HTTP method and path match.

What you learned today

Topic	Core idea
<code>req</code>	Represents the incoming HTTP request and provides information about what the client requested.
<code>res</code>	Represents the response object used by the server to send data back to the client.
<code>req.method</code>	Gives the HTTP method of the incoming request, such as GET or POST.
<code>req.url</code>	Gives the requested URL/path information.
<code>res.send()</code>	Sends a response to the client. It does not automatically stop JavaScript execution.
Route matching	The registered HTTP method and path must match the incoming request for that route handler to execute.

1. The Request Object: `req`

When a client such as a browser sends an HTTP request to your Express server, Express gives your route handler a request object. By convention, we name this object `req`.

```
app.get("/hello", (req, res) => {
  console.log(req.method);
  console.log(req.url);

  res.send("Hello Bhavana");
});
```

Memory aid: `req` = incoming. It represents information about what the client sent/requested.

The request object contains much more information than the two properties used today. Later in the course, you will learn things such as `req.params`, `req.query`, `req.body`, and `req.headers`.

2. `req.method`

`req.method` tells you which HTTP method the client used for the incoming request.

```
app.get("/hello", (req, res) => {
  console.log(req.method);
  res.send("Hello");
});
```

When you visit `http://localhost:3000/hello` in a normal browser navigation, the browser sends a GET request, so the terminal will show:

GET

Examples of HTTP methods: GET, POST, PUT, PATCH, DELETE. We will study these methods more deeply in later Express days.

3. req.url

`req.url` gives the requested URL/path information available on the incoming request.

```
app.get("/hello", (req, res) => {  
  console.log(req.url);  
  res.send("Hello");  
});
```

For a request to `/hello`, the value observed for `req.url` is `/hello`. A URL can also contain a query string, for example `/hello?test=1`. Query handling itself will be studied later.

4. The Response Object: res

The `res` parameter represents the response object. It is provided by Express to the route handler so the server can send a response back to the client.

```
app.get("/hello", (req, res) => {  
  res.send("Hello Bhavana");  
});
```

Memory aid: `res` = outgoing. It is used by the server to send the response back to the client.

A simple mental model is:



5. res.send()

res.send() sends a response to the client. In the examples today, the response is a string that the browser displays.

```
app.get("/hello", (req, res) => {  
  res.send("Hello Bhavana");  
});
```

After visiting /hello, the browser receives the response and displays Hello Bhavana.

Important distinction:

console.log() prints information in the server terminal.

res.send() sends a response to the client/browser.

6. Does res.send() stop JavaScript execution?

No. res.send() sends the HTTP response, but it does not automatically stop JavaScript execution inside the route handler.

```
app.get("/test", (req, res) => {  
  console.log("Request received");  
  res.send("Hello Bhavana");  
  console.log("After send");  
});
```

When /test is requested, the browser receives Hello Bhavana, and the terminal can still print both messages:

```
Request received  
After send
```

Key lesson: Sending the response and stopping JavaScript execution are two different ideas. If you later need to stop execution immediately after sending a response, you may use a return statement, but the important Day 2 concept is that res.send() itself does not automatically stop execution.

7. Route Matching: Method + Path

A route is selected based on the HTTP method and path. Both need to match the incoming request for the registered route handler to execute.

```
app.get("/hello", (req, res) => {  
  res.send("Hello");  
});
```

This route represents:

GET + /hello + handler

If the client sends GET /hello:

```
Method → GET      ■ matches  
Path   → /hello   ■ matches
```

Result → handler executes

But if the client sends POST /hello:

Method → POST ■ does not match GET
Path → /hello ■ matches

Result → this `app.get()` handler does not execute

Remember: Matching the path alone is not enough. The HTTP method must match too.

8. Complete Request → Route → Handler → Response Flow

1. Client sends an HTTP request
↓
2. Express receives the request
↓
3. Express checks registered routes
↓
4. HTTP method + path match
↓
5. Route handler executes with (req, res)
↓
6. Handler reads request information through req if needed
↓
7. Handler uses res to send a response
↓
8. Client receives the response

Example:

```
app.get("/hello", (req, res) => {  
  console.log(req.method);  
  console.log(req.url);  
  res.send("Hello Bhavana");  
});
```

Browser requests GET /hello → Express finds the matching GET route → handler receives req and res → req.method gives GET → req.url gives /hello → res.send() sends the response back.

9. Practical Day 2 Code

This is the type of Express application you built and tested during Day 2:

```
const express = require("express");

const app = express();

// Route = HTTP method + path + handler
app.get("/hello", (req, res) => {
  console.log(req.method);
  console.log(req.url);

  res.send("Hello Jon");
});

app.get("/skills", (req, res) => {
  res.send("I am learning backend development");
});

app.listen(3000);
```

You tested the routes in the browser and observed the request information in the terminal. You also tested the behavior of code after `res.send()`.

10. Common Beginner Mistakes

Mistake	Reality
Thinking req is the response	req represents the incoming request; res is used for the outgoing response.
Thinking res is just response text	res is the response object. <code>res.send()</code> is one way to send data using it.
Thinking <code>res.send()</code> stops all code	<code>res.send()</code> sends the response, but JavaScript execution can continue afterward.
Thinking only the path matters	The HTTP method and path both need to match the registered route.
Thinking <code>console.log()</code> responds to the browser	<code>console.log()</code> prints in the server terminal; <code>res.send()</code> sends to the client.

11. Day 2 Assessment — Reference Answers

These are the concepts you demonstrated successfully at the end of Day 2.

1. What does req represent?

The incoming HTTP request from the client. It provides information about what the client requested.

2. What does res represent?

The response object used by the Express server to send a response back to the client.

3. What does req.method give us?

It gives the HTTP method of the incoming request, such as GET, POST, PUT, PATCH, or DELETE.

4. What does req.url give us?

It gives the requested URL/path information, such as /hello or /home.

5. Does `res.send()` automatically stop JavaScript execution?

No. It sends the HTTP response, but code after it can still execute inside the handler.

6. Why doesn't `app.get('/hello')` handle `POST /hello`?

Because both the HTTP method and path need to match. The route expects `GET /hello`, while the incoming request is `POST /hello`.

12. Day 2 Quick Cheat Sheet

Expression	Meaning
<code>req</code>	Incoming request object
<code>res</code>	Response object
<code>req.method</code>	HTTP method used by the client
<code>req.url</code>	Requested URL/path information
<code>res.send(data)</code>	Sends a response to the client
<code>app.get(path, handler)</code>	Registers a GET route
<code>GET + path</code>	Must match the registered GET route for its handler to execute

13. Rebuild Day 2 Years Later

If you forget Express years from now:

1. Create/import Express.
2. Create the app with `express()`.
3. Register a route with an HTTP method and path.
4. Express calls the route handler when the incoming method + path match.
5. Use `req` to inspect incoming request information.
6. Use `res` to send the response.
7. Remember that `res.send()` sends the response but does not itself stop JavaScript execution.

Day 2 status

EXPRESS.JS DAY 2 — COMPLETE ☐

You practiced the concepts in code, tested them in the browser/terminal, and passed the Day 2 assessment.